

PROJET EN BINÔME · ÉNONCÉ ÉTUDIANT

TP3 — Spring PetClinic : concevoir, justifier et déployer sur AWS

Analyser · proposer une architecture · défendre ses choix · implémenter en compte AWS

MODULE TP3	DURÉE Projet · ~8h	MODALITÉ Binôme (2 étud.)	CERTIF. SAA-C03 · tous piliers
---------------	-----------------------	------------------------------	-----------------------------------

Instructeur : Baye Sabarane LAM — Cloud Infrastructure Engineer

Année académique 2024-2025

1. Fiche du projet

Objectif	Prendre l'application Spring PetClinic comme cas d'usage, analyser ses besoins, concevoir une architecture AWS conforme aux bonnes pratiques, justifier et défendre les choix techniques, puis l'implémenter réellement dans un compte AWS.
Durée & modalité	Projet d'environ 8 heures, en binôme (2 étudiants). Soutenance orale de défense en fin de projet.
Prérequis	TP1 et TP2 complétés. Maîtrise des 5 piliers Well-Architected, des services AWS de base (VPC, EC2/ECS/EKS, RDS, IAM, ELB) et notions d'IaC (Terraform de préférence).
Outils	draw.io · compte AWS (sandbox/pédagogique) · AWS CLI · Terraform ou CloudFormation/CDK · Git · Docker
Livrables	Schéma d'architecture (draw.io) + note de justification + dépôt Git de l'IaC + URL de l'application déployée + démo/soutenance + preuve de nettoyage des ressources.

2. Contexte & application

Spring PetClinic est l'application de démonstration officielle de l'écosystème Spring : une application web Java (Spring Boot) de gestion d'une clinique vétérinaire (propriétaires, animaux, vétérinaires, visites). Elle est représentative d'un grand nombre d'applications d'entreprise « 3-tiers » : une interface web, une couche métier et une base de données relationnelle. Votre mission est de la faire passer d'un simple JAR exécuté en local à une architecture de production sur AWS, hautement disponible et sécurisée.

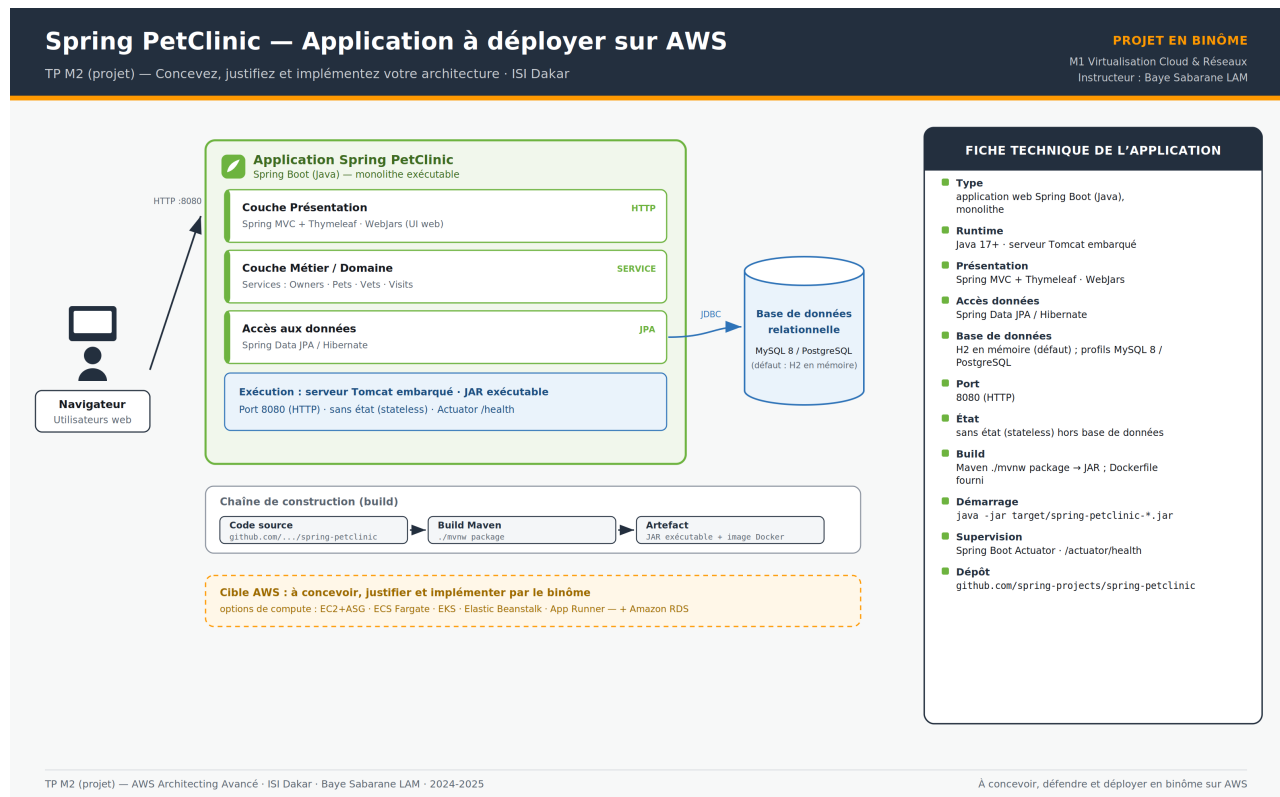


Figure 1 — Architecture logique de l'application Spring PetClinic (cas d'usage à déployer).

3. Démarrage de l'application en local

Avant toute conception, prenez en main l'application en local afin de comprendre son comportement, son port, sa base de données et son packaging :

```
git clone https://github.com/spring-projects/spring-petclinic.git
cd spring-petclinic
./mvnw package                # build Maven -> JAR exécutable
java -jar target/*.jar        # puis ouvrir http://localhost:8080

# Variante conteneur
docker build -t petclinic .    # image Docker à partir du Dockerfile fourni

# Profil base de données externe (exemple MySQL)
java -jar target/*.jar --spring.profiles.active=mysql
```

Points à observer : l'application écoute sur le port 8080, elle est sans état (stateless) hormis la base de données, et la base par défaut (H2 en mémoire) devra être remplacée par une base managée en production.

4. Travail demandé

Partie A — Analyse de l'application

Caractérisez l'application afin de fonder vos décisions d'architecture. Pour chaque point, justifiez en une à deux phrases :

- découpage en tiers (présentation, métier, données) et nature du composant à exécuter (JAR / conteneur) ;
- état applicatif : qu'est-ce qui est stateless ? que faut-il externaliser (base, fichiers, sessions) ?
- persistance : choix MySQL ou PostgreSQL, et conséquences sur le service managé visé (RDS) ;
- besoins de disponibilité, de montée en charge et de tolérance aux pannes ;
- gestion de la configuration et des secrets (mot de passe de base de données, variables d'environnement) ;
- surface de sécurité : exposition réseau, accès d'administration, chiffrement.

Partie B — Proposition d'architecture AWS

Sur draw.io (icônes AWS officielles), concevez l'architecture cible. Vous devez :

- envisager au moins deux options de calcul parmi : EC2 + Auto Scaling, ECS Fargate, EKS, Elastic Beanstalk, App Runner ;
- retenir une option et la dessiner complètement (réseau, sous-réseaux publics/privés, ALB, RDS, NAT, IAM, monitoring) ;
- respecter l'ensemble des exigences techniques minimales de la section 5 ;
- annoter chaque composant avec le(s) pilier(s) Well-Architected qu'il adresse.

Partie C — Justification & défense des choix

Rédigez une note de justification (2 à 3 pages) qui défend votre architecture. Construisez un tableau comparatif de vos options selon, au minimum, les critères suivants :

- coût (mise en place et exploitation) ; résilience / haute disponibilité ;
- scalabilité et performance ; complexité opérationnelle (effort d'exploitation) ;
- sécurité ; rapidité de mise en œuvre (time-to-market) ; adéquation aux compétences du binôme.

Concluez en expliquant pourquoi l'option retenue est la meilleure pour ce cas d'usage, et quels compromis vous assumez. Cette note sera défendue à l'oral (voir section 8).

Partie D — Implémentation dans un compte AWS (binôme)

- Déployez réellement l'architecture retenue dans un compte AWS, à deux.
- Décrivez l'infrastructure en code (Terraform de préférence, ou CloudFormation/CDK) — le déploiement doit être reproductible.
- L'application doit être accessible via une URL publique (ALB) et connectée à la base RDS.
- Préparez une démonstration : création d'un propriétaire/animal, consultation des vétérinaires, page de santé Actuator.
- À la fin, détruisez l'ensemble des ressources (terraform destroy) et fournissez-en la preuve, pour éviter tout coût résiduel.

5. Exigences techniques minimales (Well-Architected)

Quelle que soit l'option de calcul retenue, l'architecture implémentée doit satisfaire toutes les exigences suivantes :

Exigence technique minimale	Pilier
Haute disponibilité Multi-AZ (au moins 2 zones de disponibilité)	Fiabilité
Application Load Balancer en façade + health checks + terminaison TLS (ACM)	Fiabilité / Sécurité
Base managée Amazon RDS (MySQL ou PostgreSQL) en Multi-AZ, sous-réseaux privés	Fiabilité
Application en sous-réseaux privés, ALB seul en public, NAT Gateway pour les sorties	Sécurité
Security Groups en chaîne (ALB → App → RDS) ; aucun port d'administration ouvert sur Internet	Sécurité
Secret de base de données dans AWS Secrets Manager / SSM Parameter Store — jamais en dur	Sécurité
Rôles IAM en moindre privilège ; aucune clé d'accès statique sur les instances ou tâches	Sécurité
Élasticité : Auto Scaling (ASG, ou service ECS/EKS auto-scalé)	Performance
Logs et métriques CloudWatch + au moins une alarme (CPU, 5xx ou santé des cibles)	Excellence op.
Chiffrement au repos (KMS) et en transit (TLS)	Sécurité
Infrastructure as Code : Terraform ou CloudFormation/CDK (déploiement reproductible)	Excellence op.
Étiquetage de coût + AWS Budgets (plafond d'alerte) + nettoyage en fin de projet	Optimisation des coûts

6. Organisation en binôme

- **Répartition suggérée** : un équipier pilote le réseau et la sécurité (VPC, sous-réseaux, SG, IAM, secrets), l'autre le calcul et le déploiement (compute, ALB, RDS, IaC, CI/CD). Chaque équipier doit toutefois comprendre l'ensemble et pouvoir le présenter.
- **Compte AWS** : travaillez dans le compte pédagogique fourni ; préfixez et étiquetez toutes vos ressources avec le nom du binôme pour les isoler et suivre les coûts.
- **Discipline FinOps** : activez un budget d'alerte, privilégiez les petits gabarits (free tier quand possible) et éteignez/détruisez ce qui n'est pas utilisé.

7. Livrables attendus

1. Schéma de l'architecture cible (draw.io, export PNG ou PDF), composants annotés par pilier.
2. Note de justification (2–3 pages) avec le tableau comparatif des options et l'argumentaire du choix.
3. Dépôt Git contenant le code d'infrastructure (IaC) et un README de déploiement.
4. URL de l'application déployée + éventuels identifiants de démonstration.
5. Démonstration (soutenance live ou courte capture vidéo) et preuve de nettoyage des ressources.

8. Soutenance & défense des choix

Chaque binôme présente son projet en 10 à 15 minutes, suivies de questions. La soutenance doit :

- résumer l'analyse de l'application et les besoins identifiés ;
- présenter l'architecture et défendre l'option retenue face aux alternatives écartées ;
- démontrer l'application réellement déployée (URL fonctionnelle) ;
- répondre aux questions du jury sur la résilience, le coût et la sécurité.

Questions de défense (exemples)

Q1. Que se passe-t-il si une zone de disponibilité tombe ? Démontrez la continuité de service.

Q2. Où est stocké le mot de passe de la base, et comment l'application y accède-t-elle sans clé en dur ?

Q3. Pourquoi avoir choisi cette option de calcul plutôt qu'une autre ? Quel est le compromis principal ?

Q4. Comment l'application monte-t-elle en charge ? Quel signal déclenche le scaling ?

Q5. Quel est le coût mensuel estimé, et comment le réduiriez-vous en environnement réel ?

9. Grille d'évaluation

Critère d'évaluation	/20	Indicateurs
Analyse de l'application	3	Caractérisation correcte des tiers, de l'état, de la persistance et des besoins
Architecture proposée & schéma draw.io	4	Pertinence, complétude, icônes AWS, annotation des piliers
Justification & défense des choix	4	Tableau comparatif, argumentaire, qualité de la soutenance orale
Implémentation AWS fonctionnelle	5	Application accessible, Multi-AZ, RDS, réseau public/privé corrects
Sécurité & bonnes pratiques	2	Secrets gérés, IAM moindre privilège, chiffrement, SG en chaîne
IaC, FinOps, nettoyage & travail d'équipe	2	IaC reproductible, budget/étiquetage, ressources détruites, contribution équilibrée
TOTAL	20	Note minimale requise : 10/20